

RESEARCH



Deep relaxation: partial differential equations for optimizing deep neural networks

Pratik Chaudhari¹, Adam Oberman^{2*} , Stanley Osher³, Stefano Soatto¹ and Guillaume Carlier⁴

*Correspondence:

adam.oberman@mcgill.ca

²Department of Mathematics
and Statistics, McGill University,
Montreal, Canada

Pratik Chaudhari and Adam
Oberman are joint first authors
Full list of author information is
available at the end of the article

Abstract

Entropy-SGD is a first-order optimization method which has been used successfully to train deep neural networks. This algorithm, which was motivated by statistical physics, is now interpreted as gradient descent on a modified loss function. The modified, or relaxed, loss function is the solution of a viscous Hamilton–Jacobi partial differential equation (PDE). Experimental results on modern, high-dimensional neural networks demonstrate that the algorithm converges faster than the benchmark stochastic gradient descent (SGD). Well-established PDE regularity results allow us to analyze the geometry of the relaxed energy landscape, confirming empirical evidence. Stochastic homogenization theory allows us to better understand the convergence of the algorithm. A stochastic control interpretation is used to prove that a modified algorithm converges faster than SGD in expectation.

Keywords: Deep learning, Partial differential equations, Stochastic gradient descent, Neural networks, Optimal control, Proximal

1 Introduction

1.1 Overview of the main results

Deep neural networks are machine learning models that have achieved remarkable success in a number of domains from visual recognition and speech, to natural language processing and robotics [40]. A rigorous understanding of the roots of this success, however, remains elusive. These models are a parametric class of functions whose parameters are found by minimizing a non-convex loss function, typically via first-order algorithms such as stochastic gradient descent (SGD) along with a variety of regularization techniques. In this paper, we study a modification of the SGD algorithm which leads to improvements in the training time and generalization error of deep neural networks.

Chaudhari et al. [13] and Baldassi et al. [4] recently introduced a modification of the loss function of a neural network $f(x)$ with $x \in \mathbb{R}^n$ called local entropy; this function is denoted by $f_\gamma(x)$. The authors in Chaudhari et al. [13] showed that this modification is effective for training modern, high-dimensional deep neural networks. Our first result is to show that $f_\gamma(x)$ is the solution of a nonlinear partial differential equation (PDE); see Sect. 3. Define

the viscous Hamilton–Jacobi PDE

$$\frac{\partial u}{\partial t} = -\frac{1}{2} |\nabla u|^2 + \frac{\beta^{-1}}{2} \Delta u, \quad \text{for } 0 < t \leq \gamma \quad (\text{viscous HJ})$$

along with initial values $u(x, 0) = f(x)$. Here we write $\Delta u = \sum_{i=1}^n \frac{\partial^2}{\partial x_i^2} u$, for the Laplacian. We show that

$$f_\gamma(x) = u(x, \gamma),$$

where $u(x, t)$ is the solution of (viscous HJ). Using standard semi-concavity estimates in Sect. 6, the PDE interpretation immediately leads to regularity results for the solutions that are consistent with empirical observations made by Chaudhari et al. [13].

Algorithmically, our starting point is the continuous-time stochastic gradient descent equation,

$$dx(t) = -\nabla f(x(t)) dt + \beta^{-1/2} dW(t) \quad (\text{SGD})$$

for $t \geq 0$ where $W(t) \in \mathbb{R}^n$ is the n -dimensional Wiener process and the coefficient β^{-1} is the variance of a stochastic estimate of the gradient $\nabla f(x)$; see Sect. 2.2. Chaudhari et al. [13] compute the gradient of local entropy $\nabla f_\gamma(x)$ by first computing the invariant measure, $\rho(y)$, of the auxiliary stochastic differential equation (SDE)

$$dy(s) = -(\gamma^{-1}y(s) + \nabla f(x(t) - y(s))) ds + \beta^{-1/2} dW(s) \quad (1)$$

then by updating $x(t)$ by averaging against this invariant measure

$$dx(t) = -\int \nabla f(x(t) - y) \rho(dy). \quad (2)$$

The effectiveness of the algorithm may be partially explained by the fact that for small value of γ , that solutions of Fokker–Planck equation associated with the SDE (1) converge exponentially quickly to the invariant measure ρ . Tools from optimal transportation [57] or functional inequalities [2] imply that there is a threshold parameter related to the semi-convexity of $f(x)$ for which the auxiliary SGD converges exponentially; we discuss this in Sect. 2.3 and Remark 6. Beyond this threshold, the auxiliary problem can be slow to converge.

In Sect. 4, we prove using homogenization of SDEs [51] that (2) recovers the gradient of the local entropy function, $u(x, \gamma)$. In other words, in the homogenization limit, (2) is equivalent to

$$dx(t) = -\nabla u(x(t), \gamma) dt, \quad (3)$$

where $u(x, \gamma)$ is the solution of (viscous HJ). Thus, smoothing the original loss function by computing the solution of the PDE directly is not practical due to the curse of dimensionality; the auxiliary SDE (1) accomplishes this naturally.

The Elastic-SGD algorithm [66] is an influential algorithm which trains multiple deep networks in parallel with obtaining faster convergence in the distributed setting. We prove in Sect. 4.3 that the Elastic-SGD is equivalent to minimizing local entropy by the

Entropy-SGD algorithm. The equivalence is a surprising result, since the former computes the average of a coupled realizations of SGD over multiple processors while the latter computes temporal averages to evaluate (2) over a single processor. This result follows naturally if the gradient dynamics is ergodic once the homogenization framework is established for Entropy-SGD.

The parameter γ can be modulated during the optimization; we call this technique scoping. If γ is large, the smoothing achieved by the PDE is larger while a small γ recovers the true gradients of the original loss function $f(x)$ toward the end. Scoping is equivalent to replacing (3) by

$$dx(t) = -\nabla u(x(t), T - t) dt,$$

where T is a termination time for the algorithm. Scoping, which was introduced as effective heuristic in Chaudhari et al. [13], is rigorously shown to be effective in Sect. 5. Additionally, we discuss its connections to nonlinear forward–backward equations, which appear in Mean Field Games; see Remark 9.

It is unusual in stochastic non-convex optimization to obtain a proof of the superiority of a particular algorithm compared to standard stochastic gradient descent. We obtain such a result in Theorem 12 of Sect. 5 where we prove an improvement (for slightly modified dynamics) in the expected value of the loss function using the local entropy dynamics as compared to that of SGD. This result is obtained using well-known techniques from stochastic optimal control theory [25] and the comparison principle for viscosity solutions. Again, while these techniques are well-established in nonlinear PDEs, we are not aware of their application in this context.

The effectiveness of the PDE regularization is illustrated in Fig. 1.

2 Background

2.1 Deep neural networks

A deep network is a nested composition of linear functions of an input datum $\xi \in \mathbb{R}^d$ and a nonlinear function σ that is applied element-wise to its arguments. The output of a neural network, denoted by $y(x; \xi)$, can be written as

$$y(x; \xi) = \sigma' \left(x^p \sigma \left(x^{p-2} \dots \sigma \left(x^1 \xi \right) \dots \right) \right); \quad (4)$$

where $x^1, \dots, x^{p-1} \in \mathbb{R}^{d \times d}$ and $x^p \in \mathbb{R}^{d \times K}$ are the parameters or “weights” of the network. We will let $x \in \mathbb{R}^n$ denote their concatenation after rearranging each one of them as a vector. The functions $\sigma(\cdot)$ are applied point-wise to each element of their argument. For instance, in a network with rectified linear units (ReLU), we have $\sigma(z) = \max(0, z)$ for $z \in \mathbb{R}^d$. A neural network is thus a function with n parameters that produces an output $y(x; \xi) \in \{1, \dots, K\}$ for each input ξ .

In supervised learning or “training” of a deep network for the task of image classification, given a finite sample of inputs and outputs $D = \{(\xi^i, y^i)\}_{i=1}^N$ (dataset), one wishes to minimize the empirical loss¹

¹The empirical loss is a sample approximation of the expected loss, $\mathbb{E}_{x \sim P} f(x)$, which cannot be computed since the data distribution P is unknown. The extent to which the empirical loss (or a regularized version thereof) approximates the expected loss relates to generalization, i.e., the value of the loss function on (“test” or “validation”) data drawn from P but not part of the training set D .

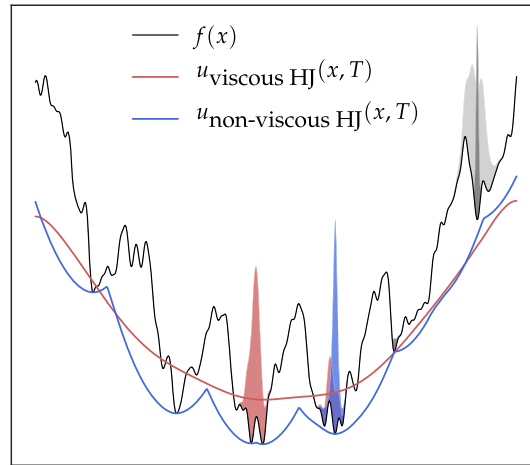


Fig. 1 One-dimensional loss function $f(x)$ (black) and smoothing of $f(x)$ by the viscous Hamilton–Jacobi equation (red) and the non-viscous Hamilton–Jacobi equation (blue). The initial density (light gray) is evolved using the Fokker–Planck equation (26) which uses solutions of the respective PDEs. The terminal density obtained using SGD (dark gray) remains concentrated in a local minimum. The function $u_{\text{viscous HJ}}(x, T)$ (red) is convex, with the global minimum close to that of $f(x)$; consequently, the terminal density (red) is concentrated near the minimum of $f(x)$. The function $u_{\text{non-viscous HJ}}(x, T)$ (blue) is non-convex, so the presence of gradients pointing away from the global minimum slows down the evolution. Nevertheless, the corresponding terminal density (blue) is concentrated in a nearly optimal local minimum. Note that $u_{\text{non-viscous HJ}}(x, T)$ can be interpreted as a generalized convex envelope of f ; it is given by the Hopf–Lax formula (HL). Also note that both the value and location of some local extrema for non-viscous HJ are unchanged, while the regions of convexity widen. On the other hand, local maxima located in smooth concave regions see their concave regions shrink, value eventually decreases. This figure was produced by solving (26) using monotone finite differences [49]

$$f(x) := \frac{1}{N} \sum_{i=1}^N f_i(x); \quad (5)$$

where $f_i(x)$ is a loss function that measures the discrepancy between the predictions $y(x, \xi^i)$ for the sample ξ^i and the true label y^i . For instance, the zero-one loss is

$$f_i(x) := \mathbf{1}_{\{y(x; \xi^i) \neq y^i\}}$$

whereas for regression, the loss could be a quadratic penalty $f_i(x) := \frac{1}{2} \|y(x; \xi^i) - y^i\|_2^2$. Regardless of the choice of the loss function, **the overall objective $f(x)$ in deep learning is a non-convex function of its argument x .**

In the sequel, we will dispense with the specifics of the loss function and consider training a deep network as a generic non-convex optimization problem given by

$$x^* = \arg \min_x f(x). \quad (6)$$

However, the particular relaxation and regularization schemes we describe are developed for, and tested on, classification tasks using deep networks.

2.2 Stochastic gradient descent (SGD)

First-order methods are the default choice for optimizing (6), since the dimensionality of x is easily in the millions. Furthermore, typically, one cannot compute the entire gradient $N^{-1} \sum_{i=1}^N \nabla f_i(x)$ at each iteration because N can also be in the millions for modern

datasets.² Stochastic gradient descent [53] consists of performing partial computations of the form

$$x^{k+1} = x^k - \eta_k \nabla f_{i_k}(x^{k-1}) \quad \text{where } \eta_k > 0, k \in \mathbb{N}, \quad (7)$$

where i_k is sampled uniformly at random from the set $\{1, \dots, N\}$, at each iteration, starting from a random initial condition x^0 . The stochastic nature of SGD arises from the approximation of the gradient using only a subset of data points. One can also average the gradient over a *set* of randomly chosen samples, called a “mini-batch.” We denote the gradient on such a mini-batch by $\nabla f_{\text{mb}}(x)$.

Variance reduction of the stochastic gradients of Sect. 2.2 leads to improved theoretical convergence rates for convex loss functions [20, 59] and is implemented in algorithms such as AdaGrad [21], SAG [59], SAGA [20], Adam [35] and others, see the review by Bottou et al. [8].

State-of-the-art results in deep learning leverage upon a number of optimization heuristics. For instance, adapting the step size, i.e., changing η_k in (7) with time k , is common practice. But while classical convergence analysis suggests step size reduction of the form $\mathcal{O}(k^{-1})$ [53, 59], piecewise constant values for the step size are common practice. Changing the mini-batch size i_k in (7) is equivalent to changing the coefficient amplitude of the noise in the gradient. As the mini-batch size goes to the size of the full data set, the coefficient of the stochastic term in (7) vanishes [43]. While stochastic gradient descent converges slower than gradient descent, i.e., without any stochasticity [59], in practice, deep networks are trained with SGD and obtain superior generalization performance only with small mini-batches [15, 28].

2.3 SGD in continuous time

Developments in this paper hinge upon interpreting SGD as a continuous-time stochastic differential equation. Along the lines of [27], we assume that the stochastic gradient $\nabla f_{\text{mb}}(x^{k-1})$ in (7) is unbiased with respect to the full gradient and has bounded variance, i.e., for all $x \in \mathbb{R}^n$,

$$\begin{aligned} \mathbb{E}[\nabla f_{\text{mb}}(x)] &= \nabla f(x), \\ \mathbb{E}[\|\nabla f_{\text{mb}}(x) - \nabla f(x)\|^2] &\leq \beta_{\text{mb}}^{-1}; \end{aligned}$$

for some $\beta_{\text{mb}}^{-1} \geq 0$. We use the notation to emphasize that the noise is coming from the mini-batch gradients. The coefficient β_{mb}^{-1} is zero if the mini-batch size is equal to the size of the dataset.

We omit the source of the noise and simply write β^{-1} while developing the theory; we will refer to the subscript mb again in Sect. 7 while providing algorithmic details. The discrete-time dynamics in (7) can be modeled by the stochastic differential equation [43]

$$dx(t) = -\nabla f(x(t)) dt + \beta^{-1/2} dW(t).$$

²For example, the ImageNet dataset [38] has $N = 1.25$ million RGB images of size 224×224 (i.e., $d \approx 10^5$) and $K = 1000$ distinct classes. A typical model, e.g., the Inception network [65] used for classification on this dataset has about $N = 10$ million parameters and is trained by running (7) for $k \approx 10^5$ updates; this takes roughly 100 hours with 8 graphics processing units (GPUs).

The generator, $\mathcal{L}$, corresponding to (SGD) is defined for smooth functions ϕ as

$$\mathcal{L}\phi = -\nabla f \cdot \nabla \phi + \frac{\beta^{-1}}{2} \Delta \phi. \quad (8)$$

The adjoint operator $\mathcal{L}^*$ is given by $\mathcal{L}^*\rho = \nabla \cdot (\nabla f \rho) + \frac{\beta^{-1}}{2} \Delta \rho$. Given the function $V(x) : X \rightarrow \mathbb{R}$, define

$$u(x, t) = \mathbb{E} \left[V(x(T)) \mid x(t) = x \right] \quad (9)$$

to be the expected value of V at the final time for the path (SGD) with initial data $x(t) = x$. By taking expectations in the Itô formula, it can be established that u satisfies the backward Kolmogorov equation

$$\frac{\partial u}{\partial t} = \mathcal{L}u, \quad \text{for } t < s \leq T,$$

along with the terminal value $u(x, T) = V(x)$. Furthermore, $\rho(x, t)$, the probability density of $x(t)$ at time t , satisfies the Fokker–Planck equation [52]

$$\frac{\partial}{\partial t} \rho(x, t) = \nabla \cdot (\nabla f(x) \rho(x, t)) + \frac{\beta^{-1}}{2} \Delta \rho(x, t) \quad (\text{FP})$$

along with the initial condition $\rho(x, 0) = \rho_0(x)$, which represents the probability density of the initial distribution. With mild assumptions on $f(x)$, in particular even if f is non-convex, $\rho(x, t)$ converges to the unique stationary solution of (FP) as $t \rightarrow \infty$ [50, Section 4.5]. Elementary calculations show that the stationary solution is the Gibbs distribution

$$\rho^\infty(x; \beta) = Z(\beta)^{-1} e^{-\beta f(x)}, \quad (10)$$

for a normalizing constant $Z(\beta)$. The notation for the parameter $\beta > 0$ comes from physics, where it corresponds to the inverse temperature. From (10), as $\beta \rightarrow \infty$, the Gibbs distribution ρ^∞ concentrates on the global minimizers of $f(x)$. Momentum is often used to accelerate convergence to this invariant distribution; this corresponds to running second-order Langevin dynamics [50, Chapter 6] given by

$$\begin{aligned} dx(t) &= p(t) dt, \\ c^{-1} dp(t) &= -(\nabla f(x(t)) + p(t)) dt + \beta^{-1/2} dW(t), \end{aligned}$$

where c is the friction coefficient. In the over-damped limit $c \rightarrow \infty$ we recover (SGD); see [64, Section 2.2.4].

The celebrated paper by Jordan et al. [34] interpreted the Fokker–Planck equation as a gradient descent in the Wasserstein metric d_{W_2} [57] of the energy functional

$$J(\rho) = \int V(x) \rho dx + \frac{\beta^{-1}}{2} \int \rho \log \rho dx.$$

If $V(x)$ is λ -convex function $f(x)$, i.e., $\nabla^2 V(x) + \lambda I$, is positive definite for all x , then the solution $\rho(x, t)$ converges exponentially in the Wasserstein metric with rate λ to ρ^∞ [11],

$$d_{W_2}(\rho(x, t), \rho^\infty) \leq d_{W_2}(\rho(x, 0), \rho^\infty) e^{-\lambda t}.$$

Let us note that a less explicit, but more general, criterion for speed of convergence in the weighted L^2 norm is the Bakry–Emery criterion which uses PDE-based estimates of the spectral gap of the potential function [2].

2.4 Smoothing of the loss function

A number of models of deep neural networks have been used in the literature to analyze characteristics of the energy landscape. If one forgoes the nonlinearities, a deep network can be seen as a generalization of principal component analysis (PCA), i.e., as a matrix factorization problem; this is explored by Baldi and Hornik [6], Haeffele and Vidal [30], Soudry and Carmon [61], Saxe et al. [58], among others. Based on empirical results such as Dauphin et al. [19], authors in Bray and Dean [9], Fyodorov and Williams [26], Choromanska et al. [17] and Chaudhari and Soatto [14] have modeled the energy landscape of deep neural networks as a high-dimensional Gaussian random field. In spite of these analyses being drawn from vastly diverse areas of machine learning, they suggest that, in general, the energy landscape of deep networks is highly non-convex and rugged.

Smoothing is an effective way to improve the performance of optimization algorithms on such a landscape, and it is our primary focus in this paper. This can be done by convolution with a kernel [16]; for specific architectures this can be done analytically [46] or by averaging the gradient over random, local perturbations of the parameters [29]. A recent paper by Li et al. [42] suggests using extensive empirical analysis that the rugged energy landscape of deep architectures can be made smoother using residual architectures, e.g., by forcing the model to be an identity mapping [31] if all the weights are zero. Our results provide a means to smoothen the energy landscape using new optimization algorithms for the same neural architecture. This paper compares and contrasts different optimization-based smoothing techniques in a unified mathematical framework.

3 PDE interpretation of local entropy

Local entropy is a modified loss function first introduced as a means for studying the energy landscape of the discrete perceptron, i.e., a “shallow” neural network with one layer and discrete parameters, e.g., $x \in \{-1, 1\}^n$ [5]. An analysis of local entropy using the replica method [45] predicts dense clusters of solutions to the optimization problem (6). Parameters that lie within such clusters yield better error on samples outside the training set D , i.e., they have improved generalization error [4].

Recent papers by Baldassi et al. [3] and Chaudhari et al. [13] have suggested algorithmic methods to exploit the above observations. The later extended the notion of local entropy to continuous variables by replacing the loss function $f(x)$ with

$$f_\gamma(x) := u(x, \gamma) = -\frac{1}{\beta} \log (G_{\beta^{-1}\gamma} * \exp(-\beta f(x))); \quad (11)$$

where $G_\gamma(x) = (2\pi\gamma)^{-d/2} \exp\left(-\frac{|x|^2}{2\gamma}\right)$ is the heat kernel.

3.1 Derivation of the viscous Hamilton–Jacobi PDE

The Cole–Hopf transformation [23, Section 4.4.1] is a classical tool used in PDEs. It relates solutions of the heat equation to those of the viscous Hamilton–Jacobi equation. For the convenience of the reader, we restate it here.

Lemma 1 (Cole–Hopf) *The local entropy function $f_\gamma(x) = u(x, \gamma)$ defined by (11) is the solution of the initial value problem for the viscous Hamilton–Jacobi equation (viscous HJ) with initial values $u(x, 0) = f(x)$. Moreover, the gradient is given by*

$$\nabla u(x, t) = \int_{\mathbb{R}^n} \frac{x-y}{t} \rho_1^\infty(dy; x) = \int_{\mathbb{R}^n} \nabla f(x-y) \rho_2^\infty(dy; x), \quad (12)$$

where $\rho_i^\infty(y; x)$ are probability distributions given by

$$\begin{aligned} \rho_1^\infty(y; x) &= Z_1^{-1} \exp\left(-\beta f(y) - \frac{\beta}{2t} |x-y|^2\right), \\ \rho_2^\infty(y; x) &= Z_2^{-1} \exp\left(-\beta f(x-y) - \frac{\beta}{2t} |y|^2\right) \end{aligned} \quad (13)$$

and $Z_i = Z_i(x)$ are normalizing constants for $i = 1, 2$.

Proof Define $u(x, t) = -\beta^{-1} \log v(x, t)$. From (11), $v = \exp(-\beta u)$ solves the heat equation

$$v_t = \frac{1}{2} \beta^{-1} \Delta v$$

with initial data $v(x, 0) = \exp(-\beta f(x))$. Taking partial derivatives gives

$$v_t = -\beta v u_t, \quad \nabla v = -\beta v \nabla u, \quad \Delta v = -\beta v \Delta u + \beta^2 v |\nabla u|^2.$$

Combining these expressions results in (viscous HJ). Differentiating $v(x, t) = \exp(-\beta u(x, t))$ using the convolution in (11), gives up to positive constants which can be absorbed into the density,

$$\begin{aligned} \nabla u(x, t) &= C \nabla_x \left(G_t * e^{-\beta f(x)} \right) = C \nabla_x \int G_t(x-y) e^{-\beta f(y)} dy \\ &= C \nabla_x \int G_t(y) e^{-\beta f(x-y)} dy. \end{aligned}$$

Evaluating the last or second to last expression for the integral leads to the corresponding parts of (12) and (13). $\square$

3.2 Hopf–Lax formula for the Hamilton–Jacobi equation and dynamics for the gradient

In addition to the connection with (viscous HJ) provided by Lemma 1, we can also explore the non-viscous Hamilton–Jacobi equation. The Hamilton–Jacobi equation corresponds to the limiting case of (viscous HJ) as the viscosity term $\beta^{-1} \rightarrow 0$.

There are several reasons for studying this equation. It has a simple, explicit formula for the gradient. In addition, semi-concavity of the solution follows directly from the Hopf–Lax formula. Moreover, under certain convexity conditions, the gradient of the solution can be computed as an exponentially convergent gradient flow. The deterministic dynamics for the gradient are a special case of the stochastic dynamics for the viscous HJ equation which are discussed in another section. Let us therefore consider the Hamilton–Jacobi equation

$$u_t = -\frac{1}{2} |\nabla u|^2. \quad (\text{HJ})$$

In the following lemma, we apply the well-known Hopf–Lax formula [23] for the solution of the HJ equation. It is also called the inf-convolution of the functions $f(x)$ and $\frac{1}{2t} |x|^2$ [10]. It is closely related to the proximal operator [47, 54].

Lemma 2 Let $u(x, t)$ be the viscosity solution of (HJ) with $u(x, 0) = f(x)$. Then

$$u(x, t) = \inf_y \left\{ f(y) + \frac{1}{2t} |x - y|^2 \right\}. \quad (\text{HL})$$

Define the proximal operator

$$\text{prox}_{tf}(x) = \arg \min_y \left\{ f(y) + \frac{1}{2t} |x - y|^2 \right\}. \quad (14)$$

If $y^* = \text{prox}_{tf}(x)$ is a singleton, then $\nabla_x u(x, t)$ exists, and

$$\nabla_x u(x, t) = \frac{x - y^*}{t} = \nabla f(y^*), \quad (15)$$

Proof The Hopf–Lax formula for the solution of the HJ equation can be found in [23]. Danskin’s theorem [7, Prop. 4.5.1] allows us to pass a gradient through an infimum, by applying it at the argminimum. The first equality is a direct application of Danskin’s Theorem to (HL). For the second equality, rewrite (HL) with $z = (x - y)/t$ and the drop the prime, to obtain,

$$u(x, t) = \inf_z \left\{ f(x - tz) + \frac{t}{2} |z|^2 \right\}.$$

Applying Danskin’s theorem to the last expression gives

$$\nabla u(x, t) = \nabla f(x - tz^*) = \nabla f(y^*).$$

□

Next we give a lemma which shows that under an auxiliary convexity condition, we can find $\nabla u(x, t)$ using exponentially convergent gradient dynamics.

Lemma 3 (Dynamics for HJ) For a fixed x , define

$$h(x, y; t) = f(x - ty) + \frac{t}{2} |y|^2.$$

Suppose that $t > 0$ is chosen so that $h(x, y; t)$ is λ -convex as a function of y (meaning that $D_y^2 h(x, y, t) - \lambda I$ is positive definite). The gradient $p = \nabla_x u(x, t)$ is then the unique steady solution of the dynamics

$$y'(s) = -\nabla_y h(x, y(s); t) = -t \left(y(s) - \nabla f(x - ty(s)) \right). \quad (16)$$

Moreover, the convergence to p is exponential, i.e.,

$$|y(s) - p| \leq |y(0) - p| e^{-\lambda s}.$$

Proof Note that (16) is the gradient descent dynamics on h which is λ -convex by assumption. Thus, by standard techniques from ordinary differential equation (ODE) (see, for example [57]) the dynamics are contraction, and the convergence is exponential. □

3.3 The invariant measure for a quadratic function

Local entropy computes a nonlinear average of the gradient in the neighborhood of x by weighing the gradients according to the steady-state measure $\rho^\infty(y; x)$.

It is useful to compute $\nabla f_\gamma(x)$ when $f(x)$ is quadratic, since this gives an estimate of the quasi-stable invariant measure near a local minimum of f . In this case, the invariant measure $\rho^\infty(y; x)$ is a Gaussian distribution.

Lemma 4 *Suppose $f(x)$ is quadratic, with $p = \nabla f(x)$ and $Q = \nabla^2 f(x)$, then the invariant measure $\rho_2^\infty(y; x)$ of (13) is a normal distribution with mean μ and covariance Σ given by*

$$\mu = x - \Sigma p \quad \text{and} \quad \Sigma = (Q + \gamma I)^{-1}.$$

In particular, for $\gamma > |Q|_2$, we have

$$\mu \approx x - (\gamma^{-1} I - \gamma^{-2} Q) p \quad \text{and} \quad \Sigma \approx \gamma^{-1} I - \gamma^{-2} Q,$$

plus higher-order terms in γ .

Proof Without loss of generality, a quadratic $f(x)$ centered at $x = 0$ can be written as

$$\begin{aligned} f(x) &= f(0) + p^\top x + \frac{1}{2} x^\top Q x, \\ \Rightarrow f(x) + \frac{\gamma}{2} x^2 &= f(0) - \mu^\top p + \frac{1}{2} (x - \mu)^\top (Q + \gamma I) (x - \mu) \end{aligned}$$

by completing the square. The distribution $\exp\left(-f(y) - \frac{1}{2\gamma} |y|^2\right)$ is thus a normal distribution with mean $\mu = x - \Sigma p$ and variance $\Sigma = (Q + \gamma I)^{-1}$ and an appropriate normalization constant. The approximation $\Sigma = \gamma^{-1} I - \gamma^{-2} Q$ which avoids inverting the Hessian follows from the Neumann series for the matrix inverse; it converges for $\gamma > |Q|_2$ and is accurate up to $\mathcal{O}(\gamma^{-3})$. $\square$

4 Derivation of local entropy via homogenization of SDEs

The Hopf–Cole formula and the corresponding formula for the gradient in Lemma 1 involve integrals which, due to the curse of dimensionality, cannot be evaluated efficiently. In this section, we obtain stochastic dynamics which through averaging (homogenization) give a computationally effective method to compute the gradient of the solution of the HJB PDE. The connection between the algorithm and the PDE was missing from Chaudhari et al. [13].

In addition, the homogenization interpretation allows us to rigorously show that an algorithm called Elastic-SGD [66] that was heuristically connected to a distributed version of local entropy in Baldassi et al. [3] is indeed equivalent to local entropy.

Homogenization is a technique used to analyze dynamical systems with multiple time-scales that have a few fast variables that may be coupled with other variables which evolve slowly. Computing averages over the fast variables allows us to obtain averaged equations for the slow variables in the limit that the time scales separate. We refer the reader to Pavliotis and Stuart [51, Chap. 10, 17] or E [22] for details. The convergence is strong: for rigorous convergence statements see Pavliotis and Stuart [51, Theorem 17.1].

4.1 Background on homogenization

Consider the system of SDEs given by

$$\begin{aligned} dx(s) &= h(x, y) ds \\ dy(s) &= \frac{1}{\varepsilon} g(x, y) ds + \frac{1}{\sqrt{\varepsilon\beta}} dW(s); \end{aligned} \quad (17)$$

where h and g are sufficiently smooth functions, $W(s) \in \mathbb{R}^n$ is the standard Wiener process, and $\varepsilon > 0$ is the homogenization parameter which introduces a fast time-scale for the dynamics of $y(s)$. Let us define the generator for the second equation to be

$$\mathcal{L}_0 = g(x, y) \cdot \nabla_y + \frac{\beta^{-1}}{2} \Delta_y.$$

We can assume that, for fixed x , the fast-dynamics, $y(s)$, has a unique invariant probability measure denoted by $\rho^\infty(y; x)$ and the operator $\mathcal{L}_0$ has a one-dimensional null-space characterized by

$$\mathcal{L}_0 1(y) = 0, \quad \mathcal{L}_0^* \rho^\infty(y; x) = 0;$$

here $1(y)$ are all constant functions in y and $\mathcal{L}_0^*$ is the adjoint of $\mathcal{L}_0$. In that case, it follows that in the limit $\varepsilon \rightarrow 0$, the dynamics for $x(s)$ in (17) converge to

$$dX(s) = \bar{h}(X) ds,$$

where the homogenized vector field for X is defined as the average against the invariant measure. Moreover, by ergodicity, in the limit, the spatial average is equal to a long term average over the dynamics independent of the initial value $y(0)$.

$$\bar{h}(X) = \int h(X, y) \rho^\infty(dy; X) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T h(x, y(s)) ds.$$

4.2 Derivation of local entropy via homogenization of SDEs

Recall (12) of Lemma 1

$$\nabla f_\gamma(x) = -\gamma^{-1} \int_{\mathbb{R}^n} (x - y) \rho_1^\infty(y; x) dy.$$

Hence, let us consider the following system of SDEs

$$\begin{aligned} dx(s) &= -\gamma^{-1} (x - y) ds, \\ dy(s) &= -\frac{1}{\varepsilon} \left[\nabla f(y) + \frac{1}{\gamma} (y - x) \right] ds + \frac{\beta^{-1/2}}{\sqrt{\varepsilon}} dW(s). \end{aligned} \quad (\text{Entropy-SGD})$$

Write

$$H(x, y; \gamma) = f(y) + \frac{1}{2\gamma} |x - y|^2.$$

The Fokker–Planck equation for the density of $y(s)$ is given by

$$\rho_t = \mathcal{L}_0^* \rho = \nabla_y \cdot (\nabla_y H \rho) + \frac{\beta^{-1}}{2} \Delta_y \rho; \quad (18)$$

The invariant measure for this Fokker–Planck equation is thus

$$\rho_1^\infty(y; x) = Z^{-1} \exp(-\beta H(x, y; \gamma)),$$

which agrees with (13) in Lemma 1.

Theorem 5 As $\varepsilon \rightarrow 0$, the system (Entropy-SGD) converges to the homogenized dynamics given by

$$dX(s) = -\nabla f_\gamma(X) ds.$$

Moreover, $-\nabla f_\gamma(x) = -\gamma^{-1} (x - \langle y \rangle)$ where

$$\langle y \rangle = \int y \rho_1^\infty(dy; X) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T y(s) ds, \quad (19)$$

where $y(s)$ is the solution of the second equation in (Entropy-SGD) for fixed x .

Proof The result follows immediately from the convergence result in Sect. 4.1 for the system (17). The homogenized vector field is

$$\bar{h}(X) = -\gamma^{-1} \int (X - y) \rho_1^\infty(y; X),$$

which is equivalent to $\nabla f_\gamma(X)$ from Lemma 1. $\square$

By Lemma 1, the following dynamics also converge to the gradient descent dynamics for f_γ .

$$\begin{aligned} dx(s) &= -\nabla f(x - y) ds \\ dy(s) &= -\frac{1}{\varepsilon} \left[\frac{y}{\gamma} - \nabla f(x - y) \right] ds + \frac{\beta^{-1/2}}{\sqrt{\varepsilon}} dW(s). \end{aligned} \quad (\text{Entropy-SGD-2})$$

Remark 6 (Exponentially fast convergence) Theorem 5 relies upon the existence of an ergodic, invariant measure $\rho_1^\infty(y; x)$. Convergence to such a measure is exponentially fast if the underlying function, $H(x, y; \gamma)$, is convex in y ; in our case, this happens if $\nabla^2 f(x) + \gamma^{-1}I$ is positive definite for all x .

4.3 Elastic-SGD as local entropy

Elastic-SGD is an algorithm introduced by Zhang et al. [66] for distributed training of deep neural networks and aims to minimize the communication overhead between a set of workers that together optimize replicated copies of the original function $f(x)$. For $n_p > 0$ distinct workers, Let $y = (y_1, \dots, y_{n_p})$, and let

$$\bar{y} = \frac{1}{n_p} \sum_{j=1}^{n_p} y_j$$

be the average value. Define the modified loss function

$$\min_x \frac{1}{n_p} \sum_{i=1}^{n_p} \left(f(x_i) + \frac{1}{2\gamma} |x_i - \bar{x}|^2 \right). \quad (20)$$

The formulation in (20) lends itself easily to a distributed optimization where worker i performs the following update, which depends only on the average of the other workers

$$dy_i(s) = -\nabla f(y_i(s))ds - \frac{1}{\gamma} (y_i(s) - \bar{y}(s))ds + \beta^{-1/2} dW_i(s). \quad (21)$$

This algorithm was later shown to be connected to the local entropy loss $f_\gamma(x)$ by Baldassi et al. [3] using arguments from replica theory. The results from the previous section can be modified to prove that the dynamics above converges to gradient descent of the local entropy. Consider the system

$$dx(s) = \frac{1}{\gamma} (x - \bar{y}) ds$$

along with (21). As in Sect. 4.2, define $H(x, y; \gamma) = \sum_{i=1}^{n_p} f(y_i) + \frac{1}{2\gamma} |x - \bar{y}|^2$. The invariant measure for the $y(s)$ dynamics corresponding to the ε scaling of (21) is given by

$$\rho^\infty(y; x) = Z^{-1} \exp \left(-\beta H(x, y; \gamma) \right),$$

and the homogenized vector field is

$$\bar{h}(X) = \gamma^{-1} \left(X - \langle \bar{y} \rangle \right),$$

where $\langle \bar{y} \rangle = \int \bar{y} \rho^\infty(d\bar{y}; X)$. By ergodicity, we can replace the average over the invariant measure with a combination of the temporal average and the average over the workers

$$\langle \bar{y} \rangle = \lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T \bar{y}(s) ds. \quad (22)$$

Thus, the same argument as in Theorem 5 applies, and we can conclude that the dynamics also converges to gradient descent for local entropy as $n_p \rightarrow \infty$.

Remark 7 (Variational Interpretation of the distributed algorithm) An alternate approach to understanding the distributed version of the algorithm, which we hope to study in the future, is modeling it as the gradient flow in the Wasserstein metric of a non-local functional. This functional is obtained by including an additional term in the Fokker–Planck functional,

$$J(\rho) = \int f_\gamma(x) \rho dx + \frac{\beta^{-1}}{2} \int \rho \log \rho dx + \frac{1}{2\gamma} \int \int |x - y|^2 \rho(x) \rho(y) dx dy;$$

see [57].

4.4 Heat equation versus the viscous Hamilton–Jacobi equation

Lemma 1 showed that local entropy is the solution to the viscous Hamilton–Jacobi equation (viscous HJ). The homogenization dynamics in (Entropy-SGD) can thus be interpreted as a way of smoothing of the original loss $f(x)$ to aid optimization. Other partial differential equations could also be used to achieve a similar effect. For instance, the following dynamics that corresponds to gradient descent on the solution of the the heat equation:

$$\begin{aligned} dx(s) &= -\nabla f(x - y) ds \\ dy(s) &= -\frac{1}{\varepsilon\gamma} y ds + \frac{1}{\sqrt{\varepsilon\beta}} dW(s). \end{aligned} \quad (\text{HEAT})$$

Using a very similar analysis as above, the invariant distribution for the fast variables is $\rho^\infty(dy; X) = G_{\beta^{-1}\gamma}(y)$. In other words, the fast dynamics for y is decoupled from x and $y(s)$ converges to a Gaussian distribution with mean zero and variance $\beta^{-1}\gamma I$. The homogenized dynamics is given by

$$dX(s) = -\nabla f(X) * G_{\beta^{-1}\gamma} ds = -\nabla v(X, \gamma), \quad (23)$$

where $v(x, \gamma)$ is the solution of the heat equation $v_t = \frac{\beta^{-1}}{2} \Delta v$ with initial data $v(x, 0) = f(x)$. The dynamics corresponds to a Gaussian averaging of the gradients.

Remark 8 (Comparison between local entropy and heat equation dynamics) The extra term in the $y(s)$ dynamics for (Entropy-SGD-2) with respect to (HEAT) is exactly the distinction between the smoothing performed by local entropy and the smoothing performed by heat equation. The latter is common in the deep learning literature under different forms, e.g., Gulcehre et al. [29] and Mobahi [46]. The former version, however, has much better empirical performance (cf. experiments in Sect. 8 and Chaudhari et al. [13]) as well as improved convergence in theory (cf. Theorem 12). Moreover, at a critical point, the gradient term vanishes, and the operators (HEAT) and (Entropy-SGD-2) coincide.

5 Stochastic control interpretation

In this section, we consider the following controlled SDE

$$dx(s) = -\nabla f(x(s)) ds - \alpha(s) ds + \beta^{-1/2} dW(s), \quad \text{for } t \leq s \leq T, \quad (\text{CSGD})$$

along with $x(t) = x$. Controlled stochastic differential equations [24, 25] are generalizations of SDEs discussed earlier in Sect. 2.3. The use of stochastic control in this context is unusual; we are not aware of any works other than Li et al. [43].

5.1 Stochastic control for a variant of local entropy

For fixed controls α , the generator for (CSGD) is given by

$$\mathcal{L}_\alpha := (-\nabla f - \alpha) \cdot \nabla + \frac{\beta^{-1}}{2} \Delta.$$

Define the cost functional for a given solution $x(\cdot)$ of (CSGD) corresponding to the control $\alpha(\cdot)$,

$$\mathcal{C}(x(\cdot), \alpha(\cdot)) = \mathbb{E} \left[V(x(T)) + \frac{1}{2} \int_t^T |\alpha(s)|^2 ds \right]. \quad (24)$$

Here the terminal cost is a given function $V : \mathbb{R}^n \rightarrow \mathbb{R}$ and we use the prototypical quadratic running cost. Define the value function

$$u(x, t) = \min_{\alpha(\cdot)} \mathcal{C}(x(\cdot), \alpha(\cdot)),$$

to be the minimum expected cost, over admissible controls, and over paths which start at $x(t) = x$. From the definition, the value function satisfies

$$u(x, T) = V(x).$$

The dynamic programming principle expresses the value function as the unique viscosity solution of a Hamilton–Jacobi–Bellman (HJB) PDE [24, 25], $u_t = H(\nabla u, D^2 u)$ where the operator H is defined by

$$H(p, Q) = \min_{\alpha} \left\{ \mathcal{L}_\alpha(p, Q) + \frac{1}{2} |\alpha|^2 \right\} := \min_{\alpha} \left\{ (-\nabla f - \alpha) \cdot p + \frac{1}{2} |\alpha|^2 + \frac{\beta^{-1}}{2} \text{tr } Q \right\}.$$

The minimum is achieved at $\alpha = p$, which results in $H(p, Q) = -\nabla f \cdot p - \frac{1}{2}|p|^2 + \frac{\beta^{-1}}{2} \operatorname{tr} Q$. The resulting PDE for the value function is

$$-u_t(x, t) = -\nabla f(x) \cdot \nabla u(x, t) - \frac{1}{2} |\nabla u(x, t)|^2 + \frac{\beta^{-1}}{2} \Delta u(x, t), \quad \text{for } t \leq s \leq T, \quad (\text{HJB})$$

along with the terminal values $u(x, T) = V(x)$. We make note that in this case, the optimal control is equal to the gradient of the solution

$$\alpha(x, t) = \nabla u(x, t). \quad (25)$$

Remark 9 (Forward-backward equations) Note that the PDE (HJB) is a terminal value problem in backwards time. This equation is well posed, and by relabeling time, we can obtain an initial value problem in forward time. The corresponding forward equation for the evolution is of a density under the dynamics (CSGD), involving the gradient of the value function. Together, these PDEs are the forward–backward system

$$\begin{aligned} -u_t &= -\nabla f \cdot \nabla u - \frac{1}{2} |\nabla u|^2 + \frac{\beta^{-1}}{2} \Delta u, \\ \rho_t &= -\nabla \cdot (\nabla u \rho) + \Delta \rho, \end{aligned} \quad (26)$$

for $0 \leq s \leq T$ along with the terminal and the initial data $u(x, T) = V(x)$, $\rho(x, 0) = \rho_0(x)$.

Remark 10 We could also obtain a stochastic control interpretation for (viscous HJ) by dropping the $\nabla f(x(s))$ term in (CSGD). Then, by a similar argument, (HJB) results in (viscous HJ). Thus, we obtain the interpretation of solutions of (viscous HJ) as the value function of an optimal control problem, minimizing the cost function (24) but with the modified dynamics. The reason for including the ∇f term in (CSGD) is that it allows us to prove the comparison principle in the next subsection.

Remark 11 (Mean field games) More generally, we can consider stochastic control problems where the control and the cost depend on the density of other players. In the case of “small” players, this can lead to mean field games [32, 39]. In the distributed setting is it natural to consider couplings (control and costs) which depend on the density (or moments of the density). In this context, it may be possible to use mean field game theory to prove an analogue of Theorem 12 and obtain an estimate of the improvement in the terminal reward.

5.2 Improvement in the value function

The optimal control interpretation above allows us to provide the following theorem for the improvement in the value function obtained by the dynamics (CSGD) using the optimal control (25), compared to stochastic gradient descent.

Theorem 12 Let $x_{\text{csgd}}(s)$ and $x_{\text{sgd}}(s)$ be solutions of (CSGD) and (SGD), respectively, with the same initial data $x_{\text{csgd}}(0) = x_{\text{sgd}}(0) = x_0$. Fix a time $t \geq 0$ and a terminal function, $V(x)$. Then

$$\mathbb{E}[V(x_{\text{csgd}}(t))] \leq \mathbb{E}[V(x_{\text{sgd}}(t))] - \frac{1}{2} \mathbb{E} \left[\int_0^t |\alpha(x_{\text{csgd}}(s), s)|^2 ds \right].$$

The optimal control is given by $\alpha(x, t) = \nabla u(x, t)$, where $u(x, t)$ is the solution of (HJB) along with terminal data $u(x, T) = V(x)$.

Proof First consider (SGD) with generator $\mathcal{L} = -\nabla f(x) \cdot \nabla + \frac{1}{2} \Delta$ given in (8). (Note the PDEs for stochastic are backward parabolic with terminal values. To be consistent with the rest of paper, we simply reverse time, which does not change the substance of the proof). The expected value function $v(x, t) = \mathbb{E}[V(x(t))]$ is the solution of the PDE $v_t = \mathcal{L}v$. Next, let $u(x, t)$ be the solution of (HJB). The PDE (HJB) says

$$u_t - \mathcal{L}u - \frac{1}{2} |\nabla u|^2 = 0.$$

so, in particular,

$$u_t - \mathcal{L}u \geq 0.$$

We also have $u(x, T) = v(x, T) = V(x)$. By the maximum principle [23] (possibly with additional technical assumptions as $|x| \rightarrow \infty$), we can conclude

$$u(x, s) \leq v(x, s), \quad \text{for all } t \leq s \leq T; \quad (27)$$

Use the definition to get

$$v(x, t) = \mathbb{E}[V(x(t)) \mid x(t) = x];$$

the expectation is taken over the paths of (SGD). Similarly,

$$u(x, t) = \mathbb{E} \left[V(x(t)) + \frac{1}{2} \int_0^t |\alpha|^2 \, ds \right]$$

over paths of (CSGD) with $x(t) = x$. Here $\alpha(\cdot)$ is the optimal control. The interpretation along with (27) gives the desired result. Note that the second term on the right hand side in Theorem 12 above can also be written as

$$\frac{1}{2} \mathbb{E} \left[\int_0^t |\nabla u(x_{\text{csgd}}(s), s)|^2 \, ds \right].$$

□

6 Regularization, widening and semi-concavity

The PDE approaches discussed in Sect. 3 result in a smoother loss function. We exploit the fact that our regularized loss function is the solution of the viscous HJ equation. This PDE allows us to apply standard semi-concavity estimates from PDE theory [10, 23] and quantify the amount of smoothing. Note that these estimates do not depend on the coefficient of viscosity, so they apply to the HJ equation as well. Indeed, semi-concavity estimates apply to solutions of PDEs in a more general setting which includes (viscous HJ) and (HJ).

We begin with special examples which illustrate the widening of local minima. In particular, we study the limiting case of the HJ equation, which has the property that local minima are preserved for short times. We then prove semi-concavity and establish a more sensitive form of semi-concavity using the harmonic mean. Finally, we explore the relationship between wider robust minima and the local entropy.

6.1 Widening for solutions of the Hamilton–Jacobi PDE

A function $f(x)$ is semi-concave with a constant C if $f(x) - \frac{C}{2} |x|^2$ is concave, refer to Cannarsa and Sinestrari [10]. Semi-concavity is a way of measuring the width of a local minimum: When a function is semi-concave with constant C , at a local minimum, no eigenvalue of the Hessian can be larger than C . The semi-concavity estimates below establish that near high curvature local minima widen faster than flat ones, when we evolve f by (viscous HJ).

It is illustrating to consider the following example.

Example 13 Let $u(x, t)$ be the solution of (viscous HJ) with $u(x, 0) = c |x|^2 / 2$. Then

$$u(x, t) = \frac{|x|^2}{2(t + c^{-1})} + \frac{\beta^{-1}n}{2} \log(t + c^{-1}).$$

In particular, in the non-viscous case, $u(x, t) = \frac{|x|^2}{2(t + c^{-1})}$.

Proof This follows from taking derivatives:

$$\begin{aligned} \nabla u(x, t) &= \frac{x}{t + c^{-1}}, & \frac{|\nabla u|^2}{2} &= \frac{|x|^2}{2(t + c^{-1})^2}, \\ \Delta u(x, t) &= \frac{n}{t + 1}, & u_t &= -\frac{|x|^2}{2(t + c^{-1})^2} + \frac{\beta^{-1}n}{2} \frac{1}{t + c^{-1}}. \end{aligned}$$

□

In the previous example, the semi-concavity constant of $u(x, t)$ is $C(t) = 1/(c^{-1} + t)$ where c measures the curvature of the initial data. This shows that for very large c , the improvement is very fast, for example $c = 10^8$ leads to $C(t = .01) \approx 10$, etc. In the sequel, we show how this result applies in general for both the viscous and the non-viscous cases. In this respect, for short times, solutions of (viscous HJ) and (HJ) widen faster than solutions of the heat equation.

Example 14 (Slower rate for the heat equation) In this example, we show that the heat equation can result in a very slow improvement of the semi-concavity. Let $v_1(x)$ be the first eigenfunction of the heat equation, with corresponding eigenvalue λ_1 , (for example $v_1(x) = \sin(x)$). In many cases λ_1 is of order 1. Then, since $v(x, t) = \exp(-\lambda_1 t) v_1(x)$ is a solution of the heat equation, the semiconcavity constant for $v(x, t)$ is $C(t) = c_0 \exp(-\lambda_1 t)$, where c_0 is the constant for $v_1(x)$. For short time, $C(t) \approx c_0(1 - \lambda_1 t)$ which corresponds to a much slower decrease in $C(t)$.

Another illustrative example is to study the widening of convex regions for the non-viscous Hamilton–Jacobi equation (HJ). As can be seen in Fig. 1, solutions of the Hamilton–Jacobi have additional remarkable properties with respect to local minima. In particular, for small values of t (which depend on the derivatives of the initial function) local minima are preserved. To motivate these properties, consider the following examples. We begin with the Burgers equation.

Example 15 (Burgers equation in one dimension) There is a well-known connection between Hamilton–Jacobi equations in one dimension and the Burgers equation [23] which is the prototypical one-dimensional conservation law. Consider the solution $u(x, t)$ of the non-viscous HJ equation (HJ) with initial value $f(x)$ where $x \in \mathbb{R}$. Let $p(x, t) = u_x(x, t)$. Then, p satisfies the Burgers equation

$$p_t = -p p_x.$$

We can solve Burgers equation using the method of characteristics, for a short time, depending on the regularity of $f(x)$. Set $p(x, t) = u_x(x, t)$. For short time, we can also use the fact that $p(x, t)$ is the fixed point of (16), which leads to

$$p(x, t) = f'(x - tp(x, t)). \quad (28)$$

In particular, the solutions of this equation remain classical until the characteristics intersect, and this time, t^* , is determined by

$$t^* f''(x - pt^*) + 1 = 0, \quad (29)$$

which recovers the convexity condition of Lemma 3.

Example 16 (Widening of convex regions in one dimension) In this example, we show that the widening of convex regions for one-dimensional displayed in Fig. 1 occurs in general. Let x be a local minimum of the smooth function $f(x)$, and let $0 \leq t < t^*$, where the critical t^* is defined by (29). Define the convexity interval as:

$I(x, t)$ = the widest interval containing x where $u(x, t)$ is convex

Let $I(x, 0) = [x_0, x_1]$, so that $f''(x) \geq 0$ on $[x_0, x_1]$ and $f''(x_0) = f''(x_1) = 0$. Then $I(x, t)$ contains $I(x, 0)$ for $0 \leq t \leq t^*$. Differentiating (28), and the solving for $p_x(x, t)$ leads to

$$\begin{aligned} p_x &= f''(x - tp) (1 - tp_x) \\ \Rightarrow p_x &= \frac{f''(x - tp)}{1 + t f''(x - tp)}. \end{aligned}$$

Since $t \leq t^*$, the last equation implies that p_x has the same sign as $f''(x - tp)$. Recall now that x_1 is an inflection point of $f(x)$. Choose x such that $x_1 = x - tp(x, t)$, in other words, $u_{xx}(x, t) = 0$. Note that $x > x_1$, since $p(x, t) > 0$. Thus, the inflection point moves to the right of x_1 . Similarly, the inflection point x_0 moves to the left. So we have established that the interval is larger.

Remark 17 In the higher-dimensional case, well-known properties of the inf-convolution can be used to establish the following facts. If $f(x)$ is convex, $u(x, t)$ given by (HL) is also convex; thus, any minimizer of f also minimizes $u(x, t)$. Moreover, for bounded $f(x)$, using the fact that $|y^*(x) - x| = \mathcal{O}(\sqrt{t})$, one can obtain a local version of the same result, viz., for short times t , a local minimum persists. Similar to the example in Fig. 1, local minima in the solution vanish for longer times.

6.2 Semi-concavity estimates

In this section, we establish semi-concavity estimates. These are well known and are included for the convenience of the reader.

Lemma 18 Suppose $u(x, t)$ is the solution of (viscous HJ), and let $\beta^{-1} \geq 0$. If

$$C_k = \sup_x u_{x_k x_k}(x, 0) \quad \text{and} \quad C_{\text{Lap}} = \sup_x \Delta u(x, 0),$$

then

$$\sup_x u_{x_k x_k}(x, t) \leq \frac{1}{C_k^{-1} + t}, \quad \text{and} \quad \sup_x \Delta u(x, t) \leq \frac{1}{C_{\text{Lap}}^{-1} + t/n}.$$

Proof 1. First consider the non-viscous case $\beta^{-1} = 0$. Define $g(x, y) = f(y) + \frac{1}{2t} |x - y|^2$ to be a function of x parametrized by y . Each $g(x, y)$, considered as a function of x is semi-concave with constant $1/t$. Then, $u(x, t)$, the solution of (HJ), which is given by the Hopf–Lax formula (HL) is expressed as the infimum of such functions. Thus $u(x, t)$ is also semi-concave with the same constant, $1/t$.

2. Next, consider the viscous case $\beta^{-1} > 0$. Let $w = u_{x_k x_k}$, differentiating (viscous HJ) twice gives

$$w_t + \nabla u \cdot \nabla w + \sum_{i=1}^n u_{x_i x_k}^2 = \frac{\beta^{-1}}{2} \Delta w. \quad (30)$$

Using $u_{x_k x_k}^2 = w^2$, we have

$$w_t + \nabla u \cdot \nabla w - \frac{\beta^{-1}}{2} \Delta w \leq -w^2.$$

Note that $w(x, 0) \leq C_k$ and the function

$$\phi(x, t) = \frac{1}{C_k^{-1} + t}$$

is a spatially independent super-solution of the preceding equation with $w(x, 0) \leq \phi(x, 0)$.

By the comparison principle applied to w and ϕ , we have

$$w(x, t) = u_{x_k x_k}(x, t) \leq \frac{1}{C_k^{-1} + t}$$

for all x and for all $t \geq 0$, which gives the first result. Now set $v = \Delta u$ and sum (30) over all k to obtain

$$v_t + \nabla u \cdot \nabla v + \sum_{i,j=1}^n u_{x_i x_j}^2 = \frac{\beta^{-1}}{2} \Delta v.$$

Thus,

$$v_t + \nabla u \cdot \nabla v - \frac{\beta^{-1}}{2} \Delta v \leq -\frac{1}{n} v^2$$

by the Cauchy–Schwartz inequality. Now apply the comparison principle to $v'(x, t) = \left(C_{\text{Lap}}^{-1} + t/n\right)^{-1}$ to obtain the second result. $\square$

6.3 Estimates on the harmonic mean of the spectrum

Our semi-concavity estimates in Sect. 6.2 gave bounds on $\sup_x u_{x_k x_k}(x, t)$ and the Laplacian $\sup_x \Delta u(x, t)$. In this section, we extend the above estimates to characterize the eigenvalue spectrum more precisely. Our approach is motivated by experimental evidence in Chaudhari et al. [13, Figure 1] and Sagun et al. [56]. The authors found that the Hessian of a typical deep network at a local minimum discovered by SGD has a very large proportion of its eigenvalues that are close to zero. This trend was observed for a variety of neural network architectures, datasets and dimensionality. For such a Hessian,

instead of bounding the largest eigenvalue or the trace (which is also the Laplacian), we obtain estimates of the harmonic mean of the eigenvalues. This effectively discounts large eigenvalues and we get an improved estimate of the ones close to zero that dictate the width of a minimum. The harmonic mean of a vector $x \in \mathbb{R}^n$ is

$$\text{HM}(x) = \left(\frac{1}{n} \sum_{i=1}^n \frac{1}{x_i} \right)^{-1}.$$

The harmonic mean is more sensitive to small values as seen by

$$\min_i x_i \leq \text{HM}(x) \leq n \min_i x_i.$$

which does not hold for the arithmetic mean. To give an example, the eigenspectrum of a typical network [13, Figure 1] has an arithmetic mean of 0.0029, while its harmonic mean is $\approx 10^{-10}$ which better reflects the large proportion of almost-zero eigenvalues.

Lemma 19 *If Λ is the vector of eigenvalues of the symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$ and $D = (a_{11}, \dots, a_{nn})$ is its diagonal vector,*

$$\text{HM}(\Lambda) \leq \text{HM}(D).$$

Proof Majorization is a partial order on vectors with nonnegative components [44, Chap. 3] and for two vectors $x, y \in \mathbb{R}^n$, we say that x is majorized by y and write $x \preceq y$ if $x = Sy$ for some doubly stochastic matrix S . The diagonal of a symmetric, positive semi-definite matrix is majorized by the vector of its eigenvalues [60]. For any Schur-concave function $f(x)$, if $x \preceq y$ we have

$$f(y) \leq f(x)$$

from Marshall et al. [44, Chap. 9]. The harmonic mean is also a Schur-concave function which can be checked using the Schur–Ostrowski criterion

$$(x_i - x_j) \left(\frac{\partial \text{HM}}{\partial x_i} - \frac{\partial \text{HM}}{\partial x_j} \right) \leq 0 \quad \text{for all } x \in \mathbb{R}_+^n$$

and we therefore have

$$\text{HM}(\Lambda) \leq \text{HM}(D).$$

□

Lemma 20 *If $u(x, t)$ is a solution of (viscous HJ) and $C \in \mathbb{R}^n$ is such that $\sup_x u_{x_k x_k}(x, t) \leq C_k$ for all $k \leq n$, then at a local minimum x^* of $u(x, t)$,*

$$\text{HM}(\Lambda) \leq \frac{1}{t + \text{HM}(C)^{-1}}$$

where Λ is the vector of eigenvalues of $\nabla^2 u(x^*, t)$.

Proof For a local minimum x^* , we have $\text{HM}(L) \leq \text{HM}(D)$ from the previous lemma; here D is the diagonal of the Hessian $\nabla^2 u(x_{\min}, t)$. From Lemma 18, we have

$$u_{x_k x_k}(x^*, t) \leq C_k(t) = \frac{1}{t + C_k^{-1}}.$$

The result follows by observing that

$$\text{HM}(C(t)) = \frac{n}{\sum_{i=1}^n C_k(t)^{-1}} = \frac{1}{t + \text{HM}(C)^{-1}}.$$

□

7 Algorithmic details

In this section, we compare and contrast the various algorithms in this paper and provide implementation details that are used for the empirical validation are provided in Sect. 8.

7.1 Discretization of (Entropy-SGD)

The time derivative in the equation for $y(s)$ in (Entropy-SGD) is discretized using the Euler–Maruyama method with time step (learning rate) η_y as

$$y^{k+1} = y^k - \eta_y \left[\nabla f(y^k) + \frac{y^k - x^k}{\gamma} \right] + \sqrt{\eta_y \beta^{-1}} \varepsilon^k, \quad (31)$$

where ε_k are zero mean, unit variance Gaussian random variables.

In practice, we do not have access to the full gradient $\nabla f(x)$ for a deep network and instead only have the gradient over a mini-batch $\nabla f_{\text{mb}}(x)$ (cf. Sect. 2.2). In this case, there are two potential sources of noise: the coefficient β_{mb}^{-1} arising from the mini-batch gradient, and any amount of extrinsic noise, with coefficient β_{ex}^{-1} . Combining these two sources leads to the equation

$$y^{k+1} = y^k - \eta_y \left[\nabla f_{\text{mb}}(y^k) + \frac{y^k - x^k}{\gamma} \right] + \sqrt{\eta_y \beta_{\text{ex}}^{-1}} \varepsilon^k, \quad (32)$$

which corresponds to (31) with

$$\beta^{-1/2} = \beta_{\text{mb}}^{-1/2} + \beta_{\text{ex}}^{-1/2}.$$

We initialize $y^k = x^k$ if k/L is an integer. The number of time steps taken for the y variable is set to be L . This corresponds to $\varepsilon = 1/L$ in (17) and to $T = 1$ in (19). This results in a discretization of the equation for x in (Entropy-SGD) given by

$$x^{k+1} = \begin{cases} x^k - \eta \gamma^{-1} (x^k - \langle y \rangle^k) & \text{if } k/L \text{ is an integer,} \\ x^k & \text{otherwise;} \end{cases} \quad (33)$$

Since we may be far from the limit $\varepsilon \rightarrow 0$, $T \rightarrow \infty$, instead of using the linear averaging for $\langle y \rangle$ in (19), we will use a forward looking average. Such an averaging gives more weight to the later steps which is beneficial for large values of γ when the invariant measure $\rho^\infty(y; x)$ in (19) does not converge quickly. The two algorithms we describe will differ in the choices of L and β_{ex}^{-1} and the definition of $\langle y \rangle$. For **Entropy-SGD**, we set $L = 20$, $\beta_{\text{ex}}^{-1} = 10^8$, and set $\langle y \rangle$ to be

$$\langle y \rangle^{k+1} = \alpha \langle y \rangle^k + (1 - \alpha) y^{k+1};$$

$\langle y \rangle^k$ is re-initialized to x^k if k/L is an integer. The parameter α is used to perform an exponential averaging of the iterates y^k . We set $\alpha = 0.75$ for experiments in Sect. 8 using Entropy-SGD. This is equivalent to the original implementation of Chaudhari et al. [13].

The step size for the y^k dynamics is fixed to $\eta_y = 0.1$. It is a common practice to perform step size annealing while optimizing deep networks (cf. Sect. 2.2), i.e., we set $\eta = 0.1$ and reduce it by a factor of 5 after every 3 epochs (iterations on the entire dataset).

7.2 Non-viscous Hamilton–Jacobi (HJ)

For the other algorithm which we denote as **HJ**, we set $L = 5$ and $\beta_{\text{ex}}^{-1} = 0$, and set $\langle y \rangle^k = y^k$ in (32) and (33), i.e., no averaging is performed and we simply take the last iterate of the y^k updates.

Remark 21 We can also construct an equivalent system of updates for **HJ** by discretizing (Entropy-SGD-2) and again setting $\beta_{\text{ex}}^{-1} = 0$; this exploits the two different ways of computing the gradient in Lemma 1 and gives

$$\begin{aligned} y^{k+1} &= (1 - \gamma^{-1} \eta_y) y^k + \eta_y \nabla f_{\text{mb}}(x^k - y^k) \\ x^{k+1} &= \begin{cases} x^k - \eta \nabla f_{\text{mb}}(x^k - y^k) & \text{if } k/L \text{ is an integer,} \\ x^k & \text{else;} \end{cases} \end{aligned} \quad (34)$$

and we initialize $y^k = 0$ if k/L is an integer.

7.3 Heat equation

We perform the update corresponding to (23)

$$x^{k+1} = x^k - \frac{\eta}{L} \left[\sum_{i=1}^L \nabla f_{\text{mb}}(x^k + \varepsilon^i) \right], \quad (35)$$

where ε^i are Gaussian random variables with zero mean and variance γI . Note that we have implemented the convolution in (23) as an average over Gaussian perturbations of the parameters x . We again set $L = 20$ for our experiments in Sect. 8.

7.4 Momentum

Momentum is a popular technique for accelerating the convergence of SGD [48]. We employ this in our experiments by maintaining an auxiliary variable z^k which intuitively corresponds to the velocity of x^k . The x^k update in (33) is modified to be

$$\begin{aligned} x^{k+1} &= z^k - \eta \nabla f_{\text{mb}}((z^k - \langle y \rangle^k)), \\ z^{k+1} &= x^k + \delta (x^k - x^{k-L}) \end{aligned}$$

if k/L is an integer; x^k and z^k are left unchanged otherwise. The other updates for x^k in (34) and (35) are modified similarly. The momentum parameter $\delta = 0.9$ is fixed for all experiments.

7.5 Scoping and tuning γ

Scoping involves setting γ to a large value and reducing it to zero as training progresses. From the point of view of regularization, sending γ to zero at the end of the algorithm makes sense, since it allows us to recover the true loss function, indeed $f_\gamma(x) \rightarrow f(x)$ as $\gamma \rightarrow 0$. On the other hand, for large values of γ , in practice, the algorithm does not correctly compute the gradient of u . However, once γ is small enough to make the auxiliary function H strongly convex, then, as explained in Remark 6, the algorithm converges exponentially fast.

For the updates in Sects. 7.1 to 7.3, we update γ every time k/L is an integer using

$$\gamma = \gamma_0 (1 - \gamma_1)^{k/L};$$



Fig. 2 MNIST

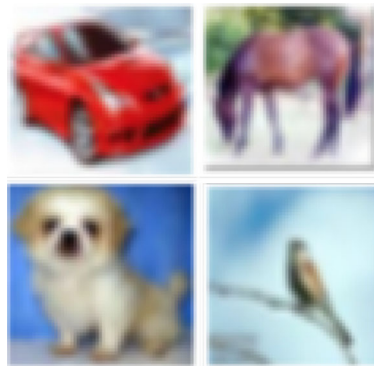


Fig. 3 CIFAR-10

we pick $\gamma_0 \in [10^{-4}, 10^{-1}]$ so as to obtain the best validation error while γ_1 is fixed to 10^{-3} for all experiments. We have obtained similar results by normalizing γ_0 by the dimensionality of x , i.e., if we set $\gamma_0 := \gamma_0/n$, this parameter becomes insensitive to different datasets and neural networks.

8 Empirical validation

We now discuss experimental results on deep neural networks that demonstrate that the PDE methods considered in this article achieve good regularization, aid optimization and lead to improved classification on modern datasets.

8.1 Setup for deep networks

In machine learning, one typically splits the data into three sets: (i) the training set D , used to compute the optimization objective (empirical loss f), (ii) the validation set, used to tune parameters of the optimization process which are not a part of the function f , e.g., mini-batch size, step size, number of iterations, and (iii) a test set, used to quantify generalization as measured by the empirical loss on previously unseen (sequestered) data. However, for the benchmark datasets considered here, it is customary in the literature to not use a separate test set. Instead, one typically reports test error on the validation set itself; for the sake of enabling direct comparison of numerical values, we follow the same practice here.

We use two standard computer vision datasets for the task of image classification in our experiments. The MNIST dataset [41] contains 70,000 gray-scale images of size 28×28 , each image portraying a hand-written digit between 0 to 9. This dataset is split into 60,000 training images and 10,000 images in the validation set. The CIFAR-10 dataset [37] contains 60,000 RGB images of size 32×32 of 10 objects (aeroplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck). The images are split as 50,000 training images and 10,000 validation images. Figures 2 and 3 show a few example images from these datasets.

8.1.1 Pre-processing

We do not perform any pre-processing for the MNIST dataset. For CIFAR-10, we perform a global contrast normalization [18] followed by a ZCA whitening transform (sometimes also called the “Mahalanobis transformation”) [37]. We use ZCA as opposed to principal component analysis (PCA) for data whitening because the former preserves visual characteristics of natural images unlike the latter; this is beneficial for convolutional neural networks. We do not perform any data augmentation, i.e., transformation of input images by mirror flips, translations, or other transformations.

8.1.2 Loss function

We defined the zero-one loss $f_i(x)$ for a binary classification problem in Sect. 2.1. For classification tasks with K classes, it is common practice to construct a network with K distinct outputs. The output $y(x; \xi^i)$ is thus a vector of length K that is further normalized to sum up to 1 using the softmax operation defined in Sect. 8.2. The cross-entropy loss typically used for optimizing deep networks is defined as:

$$f_i(x) := - \sum_{k=1}^K \mathbf{1}_{\{y^i=k\}} \log y(x; \xi^i)_k.$$

We use the cross-entropy loss for all our experiments.

8.1.3 Training procedure

We run the following algorithms for each neural network; their algorithmic details are given in Sect. 7. The ordering below corresponds to the ordering in the figures.

- **Entropy-SGD:** the algorithm of Chaudhari et al. [13] described in Sect. 7.1,
- **HEAT:** smoothing by the heat equation described in Sect. 7.3,
- **HJ:** updates for the non-viscous Hamilton–Jacobi equation described in Sect. 7.2,
- **SGD:** stochastic gradient descent given by (7),

All the above algorithms are stochastic in nature; in particular, they sample a mini-batch of images randomly from the training set at each iteration before computing the gradient. A significant difference between **Entropy-SGD** and **HJ** is that the former adds additional noise, while the latter only uses the intrinsic noise from the mini-batch gradients. We therefore report and plot the mean and standard deviation across 6 distinct random seeds for each experiment. The mini-batch size ($|i_t|$ in (7)) is fixed at 128. An “epoch” is defined to be one pass over the entire dataset; for instance, it consists of $\lceil 60,000/128 \rceil = 469$ iterations of (7) for MNIST with each mini-batch consisting of 128 different images.

We use SGD as a baseline for comparing the performance of the above algorithms in all our experiments.

Remark 22 (Number of gradient evaluations) A deep network evaluates the average gradient over an entire mini-batch of 128 images in one single back-propagation operation [55]. We define the number of back-props per weight update by L . SGD uses one back-prop per update (i.e., $L = 1$) while we use $L = 20$ back-props for Entropy-SGD (cf. Sect. 7.1), $L = 5$ back-props for the heat equation (cf. Sect. 7.3), and $L = 5$ back-props for the non-viscous HJ equation (cf. Sect. 7.2). For each of the algorithms, we plot the training loss or the validation error against the number of “effective epochs,” i.e., the number of epochs is multiplied by L which gives us a uniform scale to measure the computational efficiency of various algorithms. This is a direct measure of the wall-clock time and is agnostic to the underlying hardware and software implementation.

8.2 MNIST

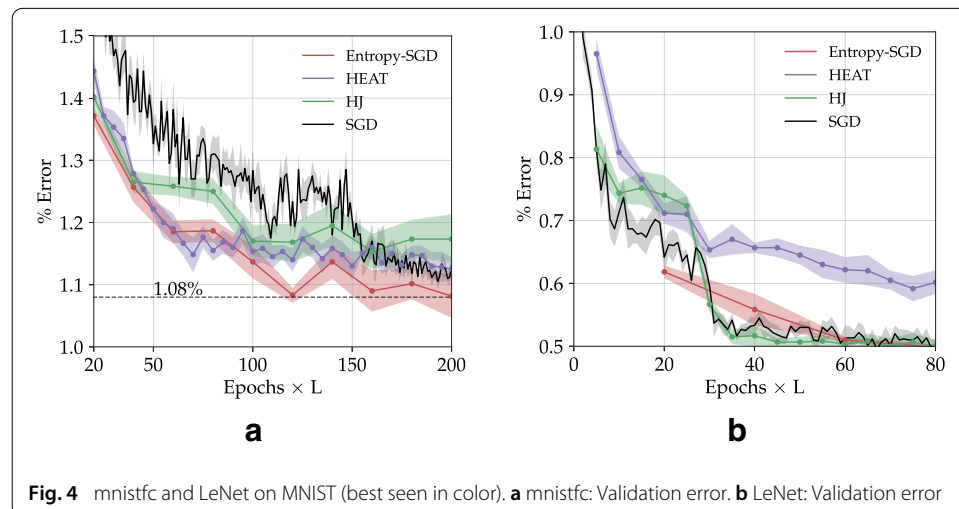
8.2.1 Fully connected network

Consider a “fully connected” network on MNIST

$$\text{mnistfc} : \text{input}_{784} \rightarrow \text{drop}_{0.2} \rightarrow \underbrace{\text{fc}_{1024} \rightarrow \text{drop}_{0.2}}_{\times 2} \rightarrow \text{fc}_{10} \rightarrow \text{softmax}.$$

The input layer reshapes each MNIST image as a vector of size 784. The notation fc_d denotes a “fully connected” dense matrix x^k in (4) with d output dimensions, followed by the ReLU nonlinearity (cf. Sect. 2.1) and an operation known as batch normalization [33] which whitens the output of each layer by subtracting the mean across a mini-batch and dividing by the standard deviation. The notation drop_p denotes the dropout layer [63] which randomly sets a fraction p of the weights to zero at each iteration; this is a popular regularization technique in deep learning, see Kingma et al. [36], Achille and Soatto [1] for a Bayesian perspective. For 10 classes in the classification problem, we create an output vector of length 10 in the last fc layer followed by softmax which picks the largest element of this vector, which is interpreted as the output (or “prediction”) by this network. The mnistfc network has $n = 1.86$ million parameters.

As Fig. 4a and Table 1 show, the final validation error for all algorithms is quite similar. The convergence rate varies; in particular, Entropy-SGD converges fastest in this case.



Note that mnistfc is a small network and the difference in the performance of the above algorithms, e.g., 1.08 % for Entropy-SGD versus 1.17 % for HJ, is minor.

8.2.2 LeNet

Our second network for MNIST is a convolutional neural network (CNN) denoted as follows:

$$\begin{aligned} \text{LeNet} : \text{input}_{28 \times 28} &\rightarrow \text{conv}_{20,5,3} \rightarrow \text{drop}_{0.25} \rightarrow \text{conv}_{50,5,2} \\ &\rightarrow \text{drop}_{0.25} \rightarrow \text{fc}_{500} \rightarrow \text{drop}_{0.25} \rightarrow \text{fc}_{10} \rightarrow \text{softmax}. \end{aligned}$$

The notation $\text{conv}_{c,k,m}$ denotes a 2D convolutional layer with c output channels, each of which is the sum of a channel-wise convolution operation on the input using a learnable kernel of size $k \times k$. It further adds ReLU nonlinearity, batch normalization and an operation known as max-pooling which down-samples the image by picking the maximum over a patch of size $m \times m$. Convolutional networks typically perform much better than fully connected ones on image classification task despite having fewer parameters, LeNet has only $n = 131,220$ parameters.

The results for LeNet are described in Fig. 4b and Table 1. This is a convolutional neural network and performs better than mnistfc on the same dataset. The final validation error is very similar for all algorithms at 0.50 % with the exception of the heat equation which only reaches 0.59 %. We can also see that the other algorithms converge quickly, in about half the number of effective epochs as that of mnistfc.

8.3 CIFAR

The CIFAR-10 dataset is more complex than MNIST and fully connected networks typically perform poorly. We will hence employ a convolutional network for this dataset. We use the All-CNN-C architecture introduced by Springenberg et al. [62] and add batch normalization:

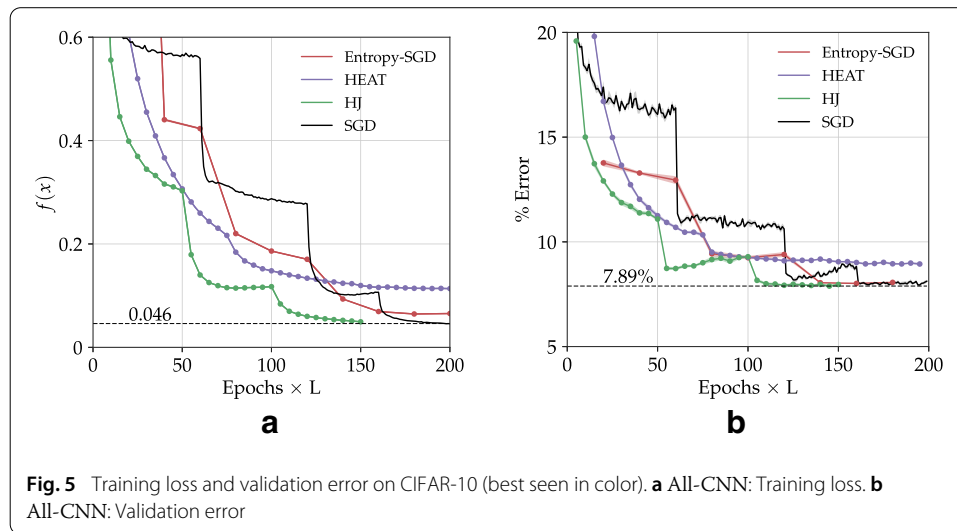
$$\begin{aligned} \text{All-CNN} : \text{input}_{3 \times 32 \times 32} &\rightarrow \text{drop}_{0.2} \rightarrow \text{block}_{96,3} \rightarrow \text{block}_{192,3} \\ &\rightarrow \underbrace{\text{conv}_{192,3}}_{\times 2} \rightarrow \text{conv}_{10} \rightarrow \text{mean-pool}_{10} \rightarrow \text{softmax}. \end{aligned}$$

where

$$\text{block}_{d,3} : \text{conv}_{d,3,1} \rightarrow \text{conv}_{d,3,1} \rightarrow \text{conv}_{d,3,1}^* \rightarrow \text{drop}_{0.5}.$$

The final convolutional layer in the above block denoted by conv^* is different from others; while they perform convolution at every pixel otherwise known as a “stride” of 1 pixel, conv^* , on the other hand, uses a stride of 2; this results in the image being down-sampled by a factor of two. Note that $\text{conv}_{c,k,m}$ with $m = 1$ does not result in any down-sampling. Max-pooling usually results in a drastic reduction of the image size and by replacing it with a strided convolution, All-CNN achieves improved performance with much fewer parameters than many other networks on CIFAR-10. The final layer denoted as mean-pool takes an input of size $10 \times 8 \times 8$ and computes the spatial average to obtain an output vector of size 10. This network has $n = 1.67$ million parameters.

Figure 5a, b shows the training loss and validation error for the All-CNN network on the CIFAR-10 dataset. The Hamilton–Jacobi equation (HJ) obtains a validation error of 7.89% in 145 epochs and thus performs best among the algorithms tested here; it also

**Table 1** Summary of experimental results: Validation error (%) @ Effective epochs

Model	Entropy-SGD	HEAT	HJ	SGD
mnistfc	1.08 ± 0.02 @ 120	1.13 ± 0.02 @ 200	1.17 ± 0.04 @ 200	1.10 ± 0.01 @ 194
LeNet	0.5 ± 0.01 @ 80	0.59 ± 0.02 @ 75	0.5 ± 0.01 @ 70	0.5 ± 0.02 @ 67
All-CNN	7.96 ± 0.05 @ 160	9.04 ± 0.04 @ 150	7.89 ± 0.07 @ 145	7.94 ± 0.06 @ 195

The first two rows correspond to low-dimensional problems. In these cases, all algorithms obtain comparable, state-of-the-art validation error; Entropy-SGD is significantly faster at training mnistfc while SGD is slightly faster for LeNet. In the largest problem on CIFAR-10, HJ has the best validation error and is the fastest. Best values are highlighted in bold.

has the lowest training cross-entropy loss of 0.046. Note that both HJ and Entropy-SGD converge faster than SGD. The heat equation performs again poorly on this dataset and has a much higher validation error than others (9.04%). This is consistent with our discussion in Sect. 4.4 which suggests that the viscous or non-viscous HJ equations result in better smoothing than the heat equation.

9 Discussion

Our results apply nonlinear PDEs, stochastic optimal control and stochastic homogenization to the analysis of two recent and effective algorithms for the optimization of neural networks. Our analysis also leads to new and improved algorithms for non-convex optimization.

We replaced the standard stochastic gradient descent (SGD) algorithm for the function $f(x)$, with SGD on the two-variable function $H(x, y; \gamma) = f(y) + \gamma^{-1}|x - y|^2/2$, along with a small parameter ε (Entropy-SGD). Using the new system, in the limit $\varepsilon \rightarrow 0$, we can provably and quantitatively improve the expected value of the original loss function. The effectiveness of our algorithm comes from the connection, via homogenization, of system of SDEs to the gradient of the function $u(x, t)$ which is the solution of the (viscous HJ) PDE with initial data $f(x)$. The function H is more convex in y than the original function f . The convexity of H in y is related to the exponentially fast convergence of the dynamics, a connection explained by the celebrated gradient flow interpretation of Fokker–Planck equation [34].

Section 4 shows that minimizing local entropy is equivalent to the influential distributed algorithm Elastic-SGD [66] if the underlying stochastic dynamics is ergodic. This insight may lead to new distributed algorithms which are presently of great interest [12].

On a practical level, training deep neural networks with a large number of hyper-parameters is very costly, in terms of both human effort and computer time. Our analysis leads to better understanding of the parameters involved in the algorithms and provides an insight into the choice of hyper-parameters for these methods. In particular, (i) the parameter γ is now understood as the time t , in the PDE (viscous HJ), (ii) scoping of γ , which was seen as a heuristic, can now be interpreted as sending a regularization parameter to zero, (iii) we set the extrinsic noise parameter β_{ex}^{-1} to zero, resulting in a simpler, more effective algorithm, and (iv) we now understand that below a critical value of γ (which is determined by the requirement that H be convex in y), the convergence of the dynamics in (Entropy-SGD) to the invariant measure is exponentially fast.

While simulated annealing and related methods work by modulating the level of noise in the dynamics, our algorithm works by modulating the smoothness of the underlying loss function. While most algorithms used in deep learning derive their motivation from the literature on convex optimization, the algorithms we have presented here are specialized to non-convex loss functions and have been shown to perform well on these problems both in theory and in practice.

Author details

¹Computer Science Department, University of California, Los Angeles, USA, ²Department of Mathematics and Statistics, McGill University, Montreal, Canada, ³Department of Mathematics and Institute for Pure and Applied Mathematics, University of California, Los Angeles, USA, ⁴CEREMADE, Université Paris IX Dauphine, Paris, France.

Acknowledgements

AO is supported by a grant from the Simons Foundation (395980); PC and SS by ONR N000141712072, AFOSR FA95501510229 and ARO W911NF151056466731CS; SO by ONR N000141410683, N000141210838, N000141712162 and DOE DE-SC00183838. AO would like to thank the hospitality of the UCLA mathematics department where this work was completed.

Received: 10 February 2018 Accepted: 9 June 2018 / Published online: 28 June 2018

References

1. Achille, A., Soatto, S.: Information dropout: learning optimal representations through noise (2016). [arXiv:1611.01353](https://arxiv.org/abs/1611.01353)
2. Bakry, D., Émery, M.: Diffusions hypercontractives. In: Séminaire de Probabilités XIX 1983/84, pp. 177–206. Springer (1985)
3. Baldassi, C., Borgs, C., Chayes, J., Ingrosso, A., Lucibello, C., Sgallietti, L., Zecchina, R.: Unreasonable effectiveness of learning neural networks: from accessible states and robust ensembles to basic algorithmic schemes. *PNAS* **113**(48), E7655–E7662 (2016a)
4. Baldassi, C., Ingrosso, A., Lucibello, C., Sgallietti, L., Zecchina, R.: Subdominant dense clusters allow for simple learning and high computational performance in neural networks with discrete synapses. *Phys. Rev. Lett.* **115**(12), 128101 (2015)
5. Baldassi, C., Ingrosso, A., Lucibello, C., Sgallietti, L., Zecchina, R.: Local entropy as a measure for sampling solutions in constraint satisfaction problems. *J. Stat. Mech. Theory Exp.* **2016**(2), 023301 (2016)
6. Baldi, P., Hornik, K.: Neural networks and principal component analysis: learning from examples without local minima. *Neural Netw.* **2**, 53–58 (1989)
7. Bertsekas, D.P., Nedić, A., Ozdaglar, A.E.: Convex analysis and optimization (2003)
8. Bottou, L., Curtis, F.E., Nocedal, J.: Optimization methods for large-scale machine learning (2016). [arXiv:1606.04838](https://arxiv.org/abs/1606.04838)
9. Bray, A.J., Dean, D.S.: Statistics of critical points of Gaussian fields on large-dimensional spaces. *Phys. Rev. Lett.* **98**(15), 150201 (2007)
10. Cannarsa, P., Sinestrari, C.: Semiconcave Functions, Hamilton–Jacobi Equations, and Optimal Control, vol. 58. Springer, Berlin (2004)
11. Carrillo, J.A., McCann, R.J., Villani, C.: Contractions in the 2-wasserstein length space and thermalization of granular media. *Arch. Ration. Mech. Anal.* **179**(2), 217–263 (2006)
12. Chaudhari, P., Baldassi, C., Zecchina, R., Soatto, S., Talwalkar, A., Oberman, A.: Parle: parallelizing stochastic gradient descent (2017). [arXiv:1707.00424](https://arxiv.org/abs/1707.00424)
13. Chaudhari, P., Choromanska, A., Soatto, S., LeCun, Y., Baldassi, C., Borgs, C., Chayes, J., Sagun, L., Zecchina, R.: Entropy-SGD: biasing gradient descent into wide valleys (2016). [arXiv:1611.01838](https://arxiv.org/abs/1611.01838)

14. Chaudhari, P., Soatto, S.: On the energy landscape of deep networks (2015). [arXiv:1511.06485](#)
15. Chaudhari, P., Soatto, S.: Stochastic gradient descent performs variational inference, converges to limit cycles for deep networks (2017) [arXiv:1710.11029](#)
16. Chen, X.: Smoothing methods for nonsmooth, nonconvex minimization. *Math. Program.* **134**(1), 71–99 (2012)
17. Choromanska, A., Henaff, M., Mathieu, M., Ben Arous, G., LeCun, Y.: The loss surfaces of multilayer networks. In: AISTATS (2015)
18. Coates, A., Lee, H., Ng, A.Y.: An analysis of single-layer networks in unsupervised feature learning. *Ann Arbor* **1001**(48109), 2 (2010)
19. Dauphin, Y., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S., Bengio, Y.: Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In: NIPS (2014)
20. Defazio, A., Bach, F., Lacoste-Julien, S.: SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives. In: NIPS (2014)
21. Duchi, J., Hazan, E., Singer, Y.: Adaptive subgradient methods for online learning and stochastic optimization. *JMLR* **12**, 2121–2159 (2011)
22. E, W.: Principles of Multiscale Modeling. Cambridge University Press, Cambridge (2011)
23. Evans, L.C.: Partial Differential Equations, volume 19 of Graduate Studies in Mathematics. American Mathematical Society (1998)
24. Fleming, W.H., Rishel, R.W.: Deterministic and Stochastic Optimal Control, vol. 1. Springer, Berlin (2012)
25. Fleming, W.H., Soner, H.M.: Controlled Markov Processes and Viscosity Solutions, vol. 25. Springer, Berlin (2006)
26. Fyodorov, Y., Williams, I.: Replica symmetry breaking condition exposed by random matrix calculation of landscape complexity. *J. Stat. Phys.* **129**(5–6), 1081–1116 (2007)
27. Ghadimi, S., Lan, G.: Stochastic first-and zeroth-order methods for nonconvex stochastic programming. *SIAM J. Optim.* **23**(4), 2341–2368 (2013)
28. Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y., He, K.: Accurate, large minibatch SGD: training Imagenet in 1 hour (2017). [arXiv:1706.02677](#)
29. Gulcehre, C., Moculski, M., Denil, M., Bengio, Y.: Noisy activation functions. In: ICML(2016)
30. Haeffele, B., Vidal, R.: Global optimality in tensor factorization, deep learning, and beyond (2015). [arXiv:1506.07540](#)
31. He, K., Zhang, X., Ren, S., Sun, J.: Identity mappings in deep residual networks (2016). [arXiv:1603.05027](#)
32. Huang, M., Malhamé, R.P., Caines, P.E., et al.: Large population stochastic dynamic games: closed-loop McKean–Vlasov systems and the Nash certainty equivalence principle. *Commun. Inf. Syst.* **6**(3), 221–252 (2006)
33. Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift (2015). [arXiv:1502.03167](#)
34. Jordan, R., Kinderlehrer, D., Otto, F.: The variational formulation of the Fokker–Planck equation. *SIAM J. Math. Anal.* **29**(1), 1–17 (1998)
35. Kingma, D., Ba, J.: Adam: A method for stochastic optimization (2014). [arXiv:1412.6980](#)
36. Kingma, D.P., Salimans, T., Welling, M.: Variational dropout and the local reparameterization trick. In: NIPS (2015)
37. Krizhevsky, A.: Learning multiple layers of features from tiny images. Master's Thesis, Computer Science, University of Toronto (2009)
38. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: NIPS (2012)
39. Lasry, J.-M., Lions, P.-L.: Mean field games. *Jpn. J. Math.* **2**(1), 229–260 (2007)
40. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**(7553), 436–444 (2015)
41. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998)
42. Li, H., Xu, Z., Taylor, G., Goldstein, T.: Visualizing the loss landscape of neural nets (2017a). [arXiv:1712.09913](#)
43. Li, Q., Tai, C., et al.: Stochastic modified equations and adaptive stochastic gradient algorithms (2017b). [arXiv:1511.06251](#)
44. Marshall, A.W., Olkin, I., Arnold, B.C.: Inequalities: Theory of Majorization and Its Applications, vol. 143. Springer, Berlin (1979)
45. Mézard, M., Parisi, G., Virasoro, M.: Spin Glass Theory and Beyond: An Introduction to the Replica Method and Its Applications, Vol. 9. World Scientific Publishing Company (1987)
46. Mobahi, H.: Training recurrent neural networks by diffusion (2016). [arXiv:1601.04114](#)
47. Moreau, J.-J.: Proximité et dualité dans un espace hilbertien. *Bull. Soc. Math. Fr.* **93**, 273–299 (1965)
48. Nesterov, Y.: A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Sov. Math. Dokl.* **27**, 372–376 (1983)
49. Oberman, A.M.: Convergent difference schemes for degenerate elliptic and parabolic equations: Hamilton–Jacobi equations and free boundary problems. *SIAM J. Numer. Anal.* **44**(2), 879–895 (2006). (electronic)
50. Pavliotis, G.A.: Stochastic Processes and Applications. Springer, Berlin (2014)
51. Pavliotis, G.A., Stuart, A.: Multiscale Methods: Averaging and Homogenization. Springer, Berlin (2008)
52. Risken, H.: Fokker–Planck equation. In: The Fokker–Planck Equation, pp. 63–95. Springer (1984)
53. Robbins, H., Monro, S.: A stochastic approximation method. *Ann. Math. Stat.* **22**(3), 400–407 (1951)
54. Rockafellar, R.T.: Monotone operators and the proximal point algorithm. *SIAM J. Control Optim.* **14**(5), 877–898 (1976)
55. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Cognit. Model.* **5**(3), 1 (1988)
56. Sagun, L., Bottou, L., LeCun, Y.: Singularity of the hessian in deep learning (2016). [arXiv:1611.07476](#)
57. Santambrogio, F.: Optimal Transport for Applied Mathematicians. Birkhäuser, New York (2015)
58. Saxe, A., McClelland, J., Ganguli, S.: Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In: ICLR (2014)
59. Schmidt, M., Le Roux, N., Bach, F.: Minimizing finite sums with the stochastic average gradient. *Math. Program.* **162**(1–2), 83–112 (2017)

60. Schur, I.: Über eine Klasse von Mittelbildungen mit Anwendungen auf die Determinantentheorie. Sitzungsberichte der Berliner Mathematischen Gesellschaft **22**, 9–20 (1923)
61. Soudry, D., Carmon, Y.: No bad local minima: Data independent training error guarantees for multilayer neural networks (2016). [arXiv:1605.08361](https://arxiv.org/abs/1605.08361)
62. Springenberg, J., Dosovitskiy, A., Brox, T., Riedmiller, M.: Striving for simplicity: the all convolutional net (2014). [arXiv:1412.6806](https://arxiv.org/abs/1412.6806)
63. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: a simple way to prevent neural networks from overfitting. JMLR **15**(1), 1929–1958 (2014)
64. Stoltz, G., Rousset, M., et al.: Free Energy Computations: A Mathematical Perspective. World Scientific, Singapore (2010)
65. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A.: Going deeper with convolutions. In: CVPR (2015)
66. Zhang, S., Choromanska, A., and LeCun, Y.: Deep learning with elastic averaging SGD. In: NIPS (2015)